

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)**

Факультет программной инженерии и компьютерных технологий

Лабораторная работа №3

Вариант № 838252

Выполнил: Федоров Д. А.

Р3206

Проверил: Кулинич Я. В.

г. Санкт-Петербург 2026 г.

СОДЕРЖАНИЕ

ЗАДАНИЕ	3
1.	3
2.	3
3.	3
4.	4
Исходный код программы:	4
Результаты работы программы:	5
Заключение:	7

Текст задания:

Лабораторная работа #3

Вариант 838252

Внимание! У разных вариантов разный текст задания!

Написать сценарий для утилиты *Apache Ant*, реализующий компиляцию, тестирование и упаковку в jar-архив кода проекта из лабораторной работы №3 по дисциплине "Веб-программирование".

Каждый этап должен быть выделен в отдельный блок сценария; все переменные и константы, используемые в сценарии, должны быть вынесены в отдельный файл параметров; MANIFEST.MF должен содержать информацию о версии и о запуске класса.

Сценарий должен реализовывать следующие цели (targets):

1. **compile** -- компиляция исходных кодов проекта.
2. **build** -- компиляция исходных кодов проекта и их упаковка в исполняемый jar-архив. Компиляцию исходных кодов реализовать посредством вызова цели **compile**.
3. **clean** -- удаление скомпилированных классов проекта и всех временных файлов (если они есть).
4. **test** -- запуск junit-тестов проекта. Перед запуском тестов необходимо осуществить сборку проекта (цель **build**).
5. **doc** -- добавление в MANIFEST.MF MD5 и SHA-1 файлов проекта, а также генерация и добавление в архив javadoc по всем классам проекта.
6. **xml** -- валидация всех xml-файлов в проекте.
7. **scp** -- переименование собранного проекта по хост на выбранный сервер по завершению сборки. Предварительно необходимо выполнить сборку проекта (цель **build**).
8. **music** -- воспроизведение музыки по завершению сборки (цель **build**).
9. **native2ascii** -- преобразование *native2ascii* для копий файлов локализации (для тестирования сценария все строковые параметры необходимо вынести из классов в файлы локализации).
10. **scp** -- осуществление получения из svn-репозитория 3 предыдущих ревизий, их сборку (по аналогии с основной) и упаковку получившихся jar-файлов в zip-архив. Сборку реализовать посредством вызова цели **build**.
11. **report** -- в случае успешного прохождения тестов сохраняет отчет junit в формате xml, добавляет его в репозиторий svn и выполняет commit.
12. **env** -- осуществляет сборку и запуск программы в альтернативных окружениях; окружение задается версией java и набором аргументов виртуальной машины в файле параметров.
13. **history** -- если проект не удастся скомпилировать (цель **compile**), загружается предыдущая версия из репозитория svn. Операция повторяется до тех пор, пока проект не удастся собрать, либо не будет получена самая первая ревизия из репозитория. Если такая ревизия найдена, то формируется файл, содержащий результат операции diff для всех файлов, измененных в ревизии, следующей непосредственно за последней работающей.
14. **diff** -- осуществляет проверку состояния рабочей копии, и, если изменения не касаются классов, указанных в файле параметров выполняет commit в репозиторий git.
15. **alt** -- создаёт альтернативную версию программы с измененными именами переменных и классов (используя задание replace/replaceAll в файлах параметров) и упаковывает её в jar-архив. Для создания jar-архива использует цель **build**.

Вопросы к защите лабораторной работы:

1. Тестирование ПО. Цель тестирования, виды тестирования.
2. Модульное тестирование, основные принципы и используемые подходы.
3. Пакет JUnit, основные API.
4. Системы автоматической сборки. Назначение, принципы работы, примеры систем.
5. Утилита make. Make-файлы, цели и правила.
6. Утилита Ant. Сценарии сборки, цели и команды.

Реализация:

<https://github.com/Dimqi/lab3opi>

Вывод:

В ходе выполнения данной лабораторной работы я улучшил навыки владения системами сборки. Узнал какое бывает тестирование, какие бывают стратегии интеграции, какие способы выбора тестового покрытия бывают и научился тестировать код при помощи JUnit.